AFRILANG Stdlib

std/c/parseyaml

Français. Bibliothèque AFRILANG ‘std/c/parseyaml‘ (niveau complex, catégorie complex). Elle expose 7 fonction(s) : yamlLen, yamlUpper, yamlLower, yamlTrim, yamlSplit, yamlJoin, yamlReplace.
‘import ”std/c/parseyaml”‘ · ‘use parseyaml‘
- ‘yamlLen(s text) → number‘ - ‘yamlUpper(s text) → text‘ - ‘yamlLower(s text) → text‘ - ‘yamlTrim(s text) → text‘ - ‘yamlSplit(s text, delim text) → list text‘ - ‘yamlJoin(parts list text, delim text) → text‘ - ‘yamlReplace(s text, from text, to text) → text‘ **English.** AFRILANG library ‘std/c/parseyaml‘ (tier complex, category complex). Exports 7 function(s): yamlLen, yamlUpper, yamlLower, yamlTrim, yamlSplit, yamlJoin, yamlReplace.
‘import ”std/c/parseyaml”‘ · ‘use parseyaml‘
- ‘yamlLen(s text) → number‘ - ‘yamlUpper(s text) → text‘ - ‘yamlLower(s text) → text‘ - ‘yamlTrim(s text) → text‘ - ‘yamlSplit(s text, delim text) → list text‘ - ‘yamlJoin(parts list text, delim text) → text‘ - ‘yamlReplace(s text, from text, to text) → text‘

Catégorie / Category	Modules complex (c/)
Niveau / Tier	complex
Fonctions / Functions	7

June 16, 2026

Contents

Français

1. Vue d'ensemble

Bibliothèque AFRILANG 'std/c/parseyaml' (niveau complex, catégorie complex). Elle expose 7 fonction(s) : yamlLen, yamlUpper, yamlLower, yamlTrim, yamlSplit, yamlJoin, yamlReplace.

'import "std/c/parseyaml"' · 'use parseyaml'

```
- `yamlLen(s text) → number`
- `yamlUpper(s text) → text`
- `yamlLower(s text) → text`
- `yamlTrim(s text) → text`
- `yamlSplit(s text, delim text) → list text`
- `yamlJoin(parts list text, delim text) → text`
- `yamlReplace(s text, from text, to text) → text`
```

2. Installation & import

Ajoutez le module à votre programme :

```
import "std/c/parseyaml"
use parseyaml
```

```
Niveau : complex · Catégorie : Modules complex (c/)
Domaines avancés — graphes, NLP, réseaux complexes.
```

3. Référence API

```
•
 yamlLen(s text) → number•
 yamlUpper(s text) → text•
 yamlLower(s text) → text•
 yamlTrim(s text) → text•
 yamlSplit(s text, delim text) → list•
 yamlJoin(parts list text, delim text) → text•
 yamlReplace(s text, from text, to text) → text
```

4. Exemples d'utilisation

```
import "std/c/parseyaml"
use parseyaml
```

```
create result = yamlLen(/* args */)
say result
```

```
create result = yamlUpper(/* args */)
say result
```

```
create result = yamlLower(/* args */)
say result
```

```
create result = yamlTrim(/* args */)
say result
```

```
create result = yamlSplit(/* args */)
say result
```

```
create result = yamlJoin(/* args */)
say result
```

5. Bonnes pratiques

```
•
 Préférez `import "std/c/parseyaml"` en tête de fichier.•
 Lancez `afrilang check` avant de livrer.•
 Combinez avec les modules stdlib de la même catégorie.•
 Voir /stdlib/ pour le source live et les démos playground.•
 Signalez les problèmes sur GitHub si une export manque.
```

6. Annexe — source

module parseyaml

```
function yamlLen(s text) returns number
end
```

```
function yamlUpper(s text) returns text
end
```

```
function yamlLower(s text) returns text
end
```

```
function yamlTrim(s text) returns text
end
```

```
function yamlSplit(s text, delim text) returns list text
end
```

```
function yamlJoin(parts list text, delim text) returns text
end
```

```
function yamlReplace(s text, from text, to text) returns text
end
end
```

English

1. Overview

AFRILANG library ‘std/c/parseyaml‘ (tier complex, category complex). Exports 7 function(s): yamlLen, yamlUpper, yamlLower, yamlTrim, yamlSplit, yamlJoin, yamlReplace.

‘import ”std/c/parseyaml”‘ · ‘use parseyaml‘

```
- `yamlLen(s text) → number`
- `yamlUpper(s text) → text`
- `yamlLower(s text) → text`
- `yamlTrim(s text) → text`
- `yamlSplit(s text, delim text) → list text`
- `yamlJoin(parts list text, delim text) → text`
- `yamlReplace(s text, from text, to text) → text`
```

2. Installation & import

Add the module to your program:

```
import "std/c/parseyaml"
use parseyaml
```

```
Tier: complex · Category: Complex modules (c/)
Advanced domains — graphs, NLP, complex networks.
```

3. API reference

```
•
 yamlLen(s text) → number•
 yamlUpper(s text) → text•
 yamlLower(s text) → text•
 yamlTrim(s text) → text•
 yamlSplit(s text, delim text) → list•
 yamlJoin(parts list text, delim text) → text•
 yamlReplace(s text, from text, to text) → text
```

4. Usage examples

```
import "std/c/parseyaml"
use parseyaml
```

```
create result = yamlLen(/* args */)
say result
```

```
create result = yamlUpper(/* args */)
say result
```

```
create result = yamlLower(/* args */)
say result
```

```
create result = yamlTrim(/* args */)
say result
```

```
create result = yamlSplit(/* args */)
say result
```

```
create result = yamlJoin(/* args */)
say result
```

5. Best practices

```
•
 Prefer `import "std/c/parseyaml"` at the top of your file.•
 Run `afrilang check` before shipping.•
 Combine with related stdlib modules from the same category.•
 See /stdlib/ for the live source and playground demos.•
 Report issues on GitHub if an export is missing or undocumented.
```

6. Appendix — source

module parseyaml

```
function yamlLen(s text) returns number
end
```

```
function yamlUpper(s text) returns text
end
```

```
function yamlLower(s text) returns text
end
```

```
function yamlTrim(s text) returns text
end
```

```
function yamlSplit(s text, delim text) returns list text
end
```

```
function yamlJoin(parts list text, delim text) returns text
end
```

```
function yamlReplace(s text, from text, to text) returns text
end
end
```

Référence rapide / Quick reference

- Import : `import "std/c/parseyaml"`
- Vérification : `afrilang check`
- Documentation : `https://afrilang.dev/stdlib/std/c/parseyaml/`

Source (extrait)

```
module parseyaml

function yamlLen(s text) returns number
end

function yamlUpper(s text) returns text
end

function yamlLower(s text) returns text
end

function yamlTrim(s text) returns text
end

function yamlSplit(s text, delim text) returns list text
end

function yamlJoin(parts list text, delim text) returns text
end

function yamlReplace(s text, from text, to text) returns text
end
end
```